Document number: P0960R3
Audience: EWG, CWG

Ville Voutilainen
Thomas Köppe
2019-02-22

Allow initializing aggregates from a parenthesized list of values

Abstract

This paper proposes allowing initializing aggregates from a parenthesized list of values; that is, `Aggr(val1, val2)` would mean the same thing as `Aggr{val1, val2}`, except that narrowing conversions are allowed. This is a language fix for the problem illustrated in N4462.

Revision history

- P0960R0: initial version
- P0960R1: wording created based on EWG feedback from Jacksonville 2018
- P0960R2: proposal changed to model an invented constructor call rather than textual transformation to a braced list
- P0960R3 (this revision): proposal changed again to talk directly about initializing elements, but together with lifetime extension rules that get close to the model of "an invented constructor". Also, array size deduction is now explicitly supported, e.g.: `int a[](1, 2, 3);`

Jacksonville 2018 discussion feedback

- Do we want more work in the direction depicted in p0960r0? 12 | 11 | 7 | 1 | 1
- Restrict the change to just dependent types? 1 | 1 | 2 | 13 | 14
- Should designated initializers be supported? 1 | 3 | 12 | 11 | 3
- Should raw arrays be supported? 4 | 14 | 11 | 2 | 1
- Incorporate ban brace-initializing an object if it would be plausible that it would use a deleted constructor? 11 | 14 | 5 | 0 | 3

Revision R1 implements supporting parenthesized initialization for aggregates including arrays, without support for designated initializers. The matter of initialization and deleted constructors is handled by separate paper(s).

Rapperswil 2018 discussion feedback

From the initial EWG discussion:

- P0960R1 as presented: 1 | 2 | 14 | 20 | 3
- P0960R1 but allowing narrowing conversions: 10 | 15 | 13 | 5 | 0

After CWG review and feedback:

- Accept CWG's recommendation to switch to a "as if by constructor" model: 18 | 15 | 9 | 2 | 1
- Make `std::vector<std::array<int, 3>>().emplace_back(1, 2, 3)` work via language specification: 3 | 1 | 20 | 11 | 11

After further CWG feedback and EWG discussion:

- Extend lifetimes of temporaries? 1 | 0 | 21 | 11 | 1
- Should evaluation occur in-order? 5 | 12 | 10 | 7 | 1
- Make aggregate initialization preferable over conversions? 1 | 2 | 15 | 11 | 3
- Make it behave *exactly* like a constructor, e.g. indeterminate evaluation order, non-extension of lifetime? 9 | 12 | 8 | 3 | 1
- Should the constructor be non-explicit? 3 | 2 | 12 | 9 | 4

This revision changes the mental model from the original "literal rewrite to a braced list" to "as if a synthesized, explicit constructor with appropriate *mem-initializer*s was called". This has the effect of allowing narrowing conversions in the parenthesized list, even when narrowing conversions would be forbidden in the corresponding braced list syntax. It also clarifies the non-extension of temporary lifetimes of temporaries bound to references, the absence of brace elision, and the absence of a well-defined order of evaluation of the arguments.

During the discussion, it was suggested by CWG that we should even break an existing corner case: Given `struct A; struct C { operator A(); }; struct A { C c; };`, the declaration `A a(c);` currently invokes `C`'s conversion function to `A`. It was suggested to change this behaviour to use the (arguably better-matching) aggregate initialization in this case, i.e. to behave like `A a{c};` and not like `A a = c;`. There was, however, no consensus to pursue this direction, and the proposal remains a pure extension at this point.

San Diego 2018 discussion feedback

In San Diego, EWG only briefly revisited the latest revision of the Rapperswil work (then D0960R2), and reconfirmed the direction of modelling an invented constructor. It was explicitly requested to ensure that direct member initializers work as expected, and that the resulting initialization can be used in constant evaluation (if possible).

Kona 2019 CWG review feedback

During CWG review in Kona, it became clear that the previously proposed "synthesized constructor" was problematic. The initially suggested constructor was (in the notation of the proposed wording):

```
explicit A(T1&& t1, ... , Tk&& tk);
```

But that constructor is inappropriate, since it does not allow non-reference members to be initialized with lvalues. A fix would be to drop the rvalue-references and instead use:

```
explicit A(T1 t1, ... , Tk tk);
```

But even if the elements are direct-initialized with `static_cast<T_i&&>(t_i)`, this approach would have required a mandatory additional move from the parameter variable, which would have removed the solution further from the design goal of being "just like brace initialization". In light of this, CWG chose to abandon the approach of an invented constructor and reverted to a direct specification of the initialization of the aggregate elements from the initializers. The issue of lifetime extension of temporaries is now addressed by explicitly adding an exception to the lifetime rules ([class.temporary, 6.6.7]/6). Moreover, the order of evaluation is again specified to be deterministic, in left-to-right order, so as to not cause undue lack of exception safety compared to brace initialization. Even though users should not specifically rely on this order, it would have been needlessly dangerous to make the behaviour different from that of brace initialization, and implementations would have been unlikely to perform evaluation in any other order anyway.

Design principles

- Any existing meaning of `A(b)` should not change.
- Parenthesized initialization and braced-initialization should be as similar as possible, but as distinct as necessary to conform with the existing mental models of braced lists and parenthesized lists.
- In particular, we have the following differences:
  - For an aggregate `A`, `A{b}` will not convert `b` to `A`, unless `b` is of type `A` or type derived from `A`. `A(b)` will convert even in other cases. This difference applies only to non-arrays.
  - Aggregate-initialization `A{x, y, x}` forbids narrowing conversions, while the new syntax `A(x, y, z)` does allow narrowing.
  - A temporary bound to a reference member of the aggregate via braces has its lifetime extended, whereas no extension happens when the temporary is passed via round parentheses.

Wording

In [class.temporary, 6.6.7]/6, add a new bullet (6.?) between (6.9) and (6.10) as follows:

> The exceptions to this lifetime rule are:
>
> - A temporary object bound to a reference parameter in a function call (7.6.1.2) persists until the completion of the full-expression containing the call.
> - A temporary object bound to a reference element of an aggregate of class type initialized from a parenthesized *expression-list* [dcl.init, 9.3] persists until the completion of the full-expression containing the *expression-list*.
> - The lifetime of a temporary bound to the returned value […]

In [dcl.init, 9.3]/(17.5), edit as follows:

> Otherwise, if the destination type is an array, the ~~program is ill-formed~~object is initialized as follows. Let x_1, …, x_k be the elements of the *expression-list*. If the destination type is an array of unknown bound, it is defined as having k elements. If k is greater than the size of the array, the program is ill-formed. Otherwise, the i^{th} array element is copy-initialized with x_i for each $1 \le i \le k$, and value-initialized for each $k < i \le n$. For each $1 \le i < j \le n$, every value computation and side effect associated with the initialization of the i^{th} element of the array is sequenced before those associated with the initialization of the j^{th} element.

In [dcl.init, 9.3]/(17.6.2), edit as follows:

> Otherwise, if the initialization is direct-initialization, or if it is copy-initialization where the cv-unqualified version of the source type is the same class as, or a derived class of, the class of the destination, constructors are considered. The applicable constructors are enumerated (16.3.1.3), and the best one is chosen through overload resolution (16.3). The constructor so selected is called to initialize the object, with the initializer expression or expression-list as its argument(s). If no constructor applies and the destination type is not an aggregate, or the overload resolution is ambiguous, the initialization is ill-formed.

After [dcl.init, 9.3]/(17.6.3) insert a new sub-bullet (17.6.?) as follows:

> Otherwise, if the destination type is a (possibly cv-qualified) aggregate class `A` and the initializer is a parenthesized *expression-list*, the object is initialized as follows. Let e_1, …, e_n be the elements of the aggregate [dcl.init.aggr, 9.3.1]. Let x_1, …, x_k be the elements of the *expression-list*. If k is greater than n, the program is ill-formed. The element e_i is copy-initialized with x_i for $1 \le i \le k$. The remaining elements are initialized with their default member initializers, if any, and otherwise are value-initialized. For each $1 \le i < j \le n$, every value computation and side effect associated with the initialization of e_i is sequenced before those associated with the initialization of e_j. [*Note*: By contrast with direct-list-initialization, narrowing conversions [dcl.init.list, 9.3.4] are permitted, designators are not permitted, a temporary object bound to a reference does not have its lifetime extended [class.temporary, 6.6.7], and there is no brace elision. [*Example*:
>
> ```
> struct A {
>   int a;
>   int&& r;
> };
>
> int f();
> int n = 10;
>
> A a1{1, f()};               // OK, lifetime is extended
> A a2(1, f());               // well-formed, but dangling reference
> A a3{1.0, 1};               // error: narrowing conversion
> A a4(1.0, 1);               // well-formed, but dangling reference
> A a5(1.0, std::move(n));    // OK
> ```
>
> *— end example*] *— end note*]

In [cpp.predefined, 14.8] Table 17, add the following feature test macro:

> `__cpp_aggregate_paren_init 201902L`

Acknowledgements

Great many thanks to the members of the Core Working Group for their thorough and diligent work across many rounds of wording review in Rapperswil and in Kona, and to Tomasz Kamiński for valuable feedback.